

Среда разработки GoLand использует статический анализатор кода, работающий с консольными утилитами форматирования `gofmt` и обновления импортируемых пакетов `goimports`. GoLand выполняет выравнивание синтаксических конструкций. Для принудительного форматирования всего файла или выделенного фрагмента применяется сочетание клавиш `Ctrl + Alt + L`. [P2-1]

При вызове в коде функции или структуры из стандартной библиотеки или внешнего модуля IDE добавляет соответствующий пакет в секцию импортов. При удалении строк, использующих пакет, неиспользуемый импорт вырезается из файла. В Go наличие объявленных, но неиспользуемых импортов запрещено спецификацией и приводит к ошибке компиляции. Автоматизацию этих операций можно закрепить за каждым сохранением файла, установить настройки `Reformat code` и `Optimize imports`. [P2-2]

Проверки кода в GoLand постоянно сканируют исходный код. Синтаксические ошибки выделяются красным цветом, логические предупреждения - желтым, стилистические замечания - серым цветом. При обнаружении дефекта курсор устанавливается на подсвеченное слово и используется комбинация `Alt + Enter`, далее нужно выбрать автоматическое исправление. [P2-3]

Пример кода до исправления

В этом коде есть нарушения отступов, подключён неиспользуемый пакет `math` и объявлена переменная `age`, которая не задействована в вычислениях.

```
package main
import (
    "fmt"
    "math"
)
func main() {
    age := 20
    fmt.Println("Hello, World!")
}
```

Использование команды `Ctrl + Alt + L` выравнивает отступы внутри функции `main`. С помощью комбинации `Alt + Enter` на строке пакета «`math`» вызывается контекстное меню, где выбирается опция оптимизации импортов, лишняя строка удаляется. На строке с переменной `age` быстрое исправление удаляет неиспользуемый идентификатор. После применения автоматических инструментов исходный код приводится к синтаксически корректному виду.

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, World!")
}
```

Процесс выполнения программ в GoLand контролируют конфигурации запуска. Среда автоматически распознает файлы, содержащие точку входа `func main()`, и позволяет запускать их без обращения к терминалу. Для быстрого запуска используется элемент на левом поле текстового редактора напротив функции `main`.

Нажатие на данную иконку или использование сочетания клавиш Ctrl + Shift + F10 создает временную конфигурацию и выполняет текущий код. [P2-4]

Настройка постоянной конфигурации необходима при передаче аргументов командной строки или переменных окружения. Конфигурация настраивается через меню путем добавления шаблона Go Build. При редактировании конфигурации необходимо отметить Run kind для определения объекта компиляции. Значение File запускает один файл, а значение Directory собирает пакет в указанной папке. Нужно также отметить Program arguments, строку параметров, передаваемых в программу. Параметры разделяются пробелами. Также следует отметить Environment, таблицу переменных окружения в формате «ключ-значение», доступных через функции стандартной библиотеки. [P2-5]

Запуск конфигурации выполняется нажатием кнопки со стрелкой на панели инструментов или комбинацией Shift + F10. При запуске в нижней части интерфейса открывается панель Run, отображающая команду сборки, вывод стандартного потока, код завершения процесса. [P2-5]

Если программа завершается со сбоем, строки вызова в консоли становятся кликабельными ссылками с путями к файлам и номерами строк кода. Переход по ссылке перемещает курсор в редакторе к строке, вызвавшей сбой. Кнопка Stop либо сочетание Ctrl + F2 позволяет принудительно завершить процесс. [P2-6]

Пример программы

```
package main
import (
    "fmt"
    "os"
)
func main() {
    args := os.Args[1:]
    if len(args) == 0 {
        fmt.Println("No arguments provided")
        return
    }
    fmt.Println("First argument:", args[0])
}
```

Вывод программы в окне Run при указании аргумента Test:

```
C:\Users\User\AppData\Local\Programs\Go\bin\go.exe          run          C:
\Users\User\GoLandProjects\GoLand-demo\main.go Test
First argument: Test
Process finished with exit code 0
```

Выполнение программы начинается с функции main, которая считывает параметры командной строки в срез args. Встроенная функция len определяет количество элементов в этом срезе, а с помощью конструкции if проверяется длина среза, если аргументы отсутствуют, выводится сообщение «No arguments provided», работа завершается. В противном случае функция fmt.Println принимает первый элемент и выводит его значение в консоль.

При запуске программы внутри среды разработки создается временный исполняемый файл в скрытом каталоге операционной системы. Он автоматически удаляется после завершения процесса. Этот режим предназначен для тестирования в процессе написания кода. [P2-7]

Компиляция переводит исходный код в постоянный исполняемый файл, содержащий машинный код. Полученный файл можно запускать на других системах без установки GoLand и инструментов языка Go. [P2-7]

Для сборки исполняемого файла используется встроенный терминал, вызываемый комбинацией клавиш Alt + F12. В командной строке вводится стандартная команда: go build -o program.exe main.go. Параметр -o позволяет задать имя файла. Если выполнить команду без дополнительных параметров, компилятор соберет проект и поместит исполняемый файл в корневой каталог текущего проекта под именем этой папки. В дереве файлов проекта отобразится элемент с расширением .exe. [P2-5]

Если программа работает без системных ошибок, но выдает неверный результат вычислений, используется интерактивная отладка. Она позволяет временно приостановить выполнение приложения в заданной строке кода, проанализировать состояние памяти и проследить за выполнением программы с помощью встроенного отладчика. [P2-8]

Точка останова указывает на приостановление выполнения до заданной строки кода. Устанавливается кликом по полю слева от номеров строк в редакторе. Шаг с обходом от текущей строки перемещает указатель на следующую, не заходя внутрь вызываемых функций. Шаг с обходом выполняется нажатием клавиши F8. Шаг со входом помогает перейти внутрь вызываемой функции для анализа ее логики. Вызывается клавишей F7. [P2-6]

Пример исправления логической ошибки

В данном коде функция вычисляет среднее арифметическое элементов массива. Функция возвращает некорректно округленный результат.

```
package main
import "fmt"
func average(numbers []int) float64 {
    total := 0
    for i := 0; i < len(numbers); i++ {
        total += numbers[i]
    }
    return float64(total / len(numbers))
}
func main() {
    numbers_list := []int{10, 20, 15, 40}
    result := average(numbers_list)
    fmt.Printf("Результат: %f\n", result)
}
```

Нужно установить точку останова на строке с вычислением среднего значения. затем запустить отладку комбинацией клавиш Shift + F9 или кнопкой Debug. При достижении точки останова необходимо проверить сумму элементов,

длину массива, промежуточный результат деления в панели Variables. Затем с помощью шагов отладки с использованием клавиш F8 и F7 нужно проверить изменение значений на каждом этапе вычислений. В данном коде выражение `return float64(total/len(numbers))` приводило к ошибке вычислений из-за деления нацело, после этого результат преобразовывался в вещественный тип. В исправленном коде выражение `return float64(total)/float64(len(numbers))` приводит операнды к типу `float64`, деление выполняется без потери точности.

```
package main
import "fmt"
func average(numbers []int) float64 {
    total := 0
    for i := 0; i < len(numbers); i++ {
        total += numbers[i]
    }
    return float64(total) / float64(len(numbers))
}
func main() {
    numbers_list := []int{10, 20, 15, 40}
    result := average(numbers_list)
    fmt.Printf("Результат: %f\n", result)
}
```

После внесения изменений программа выводит результат 21.250000.